

BÀI 8: KIỂM THỬ DÒNG DỮ LIỆU

Mục tiêu:

- **Về kiến thức:** cung cấp cho sinh viên kiến thức về kỹ thuật kiểm thử dòng dữ liệu: các vấn đề phổ biến về dòng dữ liệu, các thành phần của đồ thị dòng dữ liệu, các độ đo kiểm thử dòng dữ liệu
- **Về kỹ năng:** giúp sinh viên vẽ được đồ thị dòng dữ liệu cho các đơn vị mã nguồn cụ thể, thiết kế testcase thoả mãn các độ đo kiểm thử

8.1 Các vấn đề phổ biến về dòng dữ liệu

Trong quá trình lập trình, các lập trình viên có thể viết các câu lệnh “bất thường” hoặc không tuân theo chuẩn lập trình. Chúng ta gọi những bất thường liên quan đến việc khai báo, khởi tạo giá trị cho các biến và sử dụng chúng là các vấn đề về dòng dữ liệu của đơn vị chương trình. Ví dụ, một lập trình viên có thể sử dụng một biến mà không khởi tạo giá trị sau khi khai báo nó (`int x; if(x == 100){ ...}`).

Các vấn đề phổ biến về dòng dữ liệu có thể được phát hiện bằng phương pháp kiểm thử dòng dữ liệu tĩnh. Theo Fosdick và Osterweil, các vấn đề này được chia thành ba loại như sau:

- **Gán giá trị rồi gán tiếp giá trị (Loại 1):** Ví dụ, Hình 8.1 chứa hai câu lệnh tuần tự `x = f1(y); x = f2(z);` với `f1` và `f2` là các hàm đã định nghĩa trước và `y, z` lần lượt là các tham số đầu vào của các hàm này. Chúng ta có thể xem xét hai câu lệnh tuần tự này với các tình huống sau:
 - Khi câu lệnh thứ hai được thực hiện, giá trị của biến `x` được gán và câu lệnh đầu không có ý nghĩa.
 - Lập trình viên có thể có nhầm lẫn ở câu lệnh đầu. Câu lệnh này có thể là gán giá trị cho một biến khác như là `w = f1(y)`.
 - Có thể có nhầm lẫn ở câu lệnh thứ hai. Lập trình viên định gán giá trị cho một biến khác như là `w = f2(z)`.
 - Một hoặc một số câu lệnh giữa hai câu lệnh này bị thiếu. Ví dụ như câu lệnh `w = f3(x)`.

```

:
:
:
x = f1(y);
x = f2(z);
:
:

```

Hình 8.1: Tuần tự các câu lệnh có vấn đề thuộc loại 1

Chỉ có lập trình viên và một số thành viên khác trong dự án mới có thể trả lời một cách chính xác vấn đề trên thuộc trường hợp nào trong bốn tình huống trên. Mặc dù vậy, những vấn đề tương tự như ví dụ này là khá phổ biến và chúng ta cần phân tích mã nguồn để phát hiện ra chúng.

- **Chưa gán giá trị nhưng được sử dụng (Loại 2):** Ví dụ, Hình 8.2 chứa ba câu lệnh tuần tự với y là một biến đã được khai báo và gán giá trị ($y = f(x_1)$). Trong trường hợp này, biến z chưa được gán giá trị khởi tạo nhưng đã được sử dụng trong câu lệnh để tính giá trị của biến x ($x = y + z$). Chúng ta cũng có thể lý giải vấn đề này theo các tình huống sau:

- Lập trình viên có thể bỏ quên lệnh gán giá trị cho biến z trước câu lệnh tính toán giá trị cho biến x . Ví dụ, $z = f_2(x_2)$, với f_2 là một hàm đã được định nghĩa và x_2 là một biến đã được khai báo và gán giá trị.
- Có thể có sự nhầm lẫn giữa biến z với một biến đã được khai báo và gán giá trị. Ví dụ, $x = y + x_2$

```

:
:
:
y=f(x1);
int z;
x=y+z;
:
:

```

Hình 8.2: Tuần tự các câu lệnh có vấn đề thuộc loại 2.

- **Đã được khai báo và gán giá trị nhưng không được sử dụng (Loại 3):** Nếu một biến đã được khai báo và gán giá trị nhưng không hề được sử dụng (trong các câu lệnh tính toán hoặc trong các biểu thức điều kiện), chúng ta cần xem xét cẩn thận vấn đề này. Tương tự như các trường hợp trên, các tình huống sau có thể được sử dụng để lý giải cho vấn đề này:

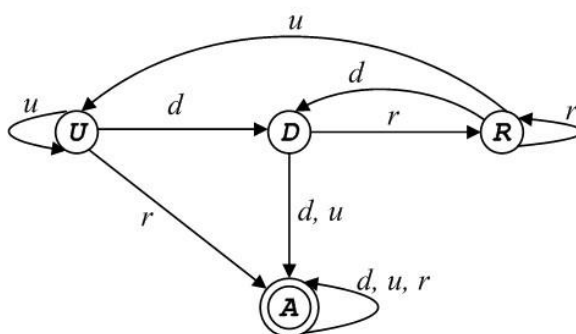
- Có sự nhầm lẫn giữa biến này và một số biến khác được sử dụng trong chương trình. Trong thiết kế, biến này được sử dụng nhưng nó đã bị thay thế (do nhầm lẫn) bởi một biến khác.
- Biến này thực sự không được sử dụng trong chương trình. Lúc đầu lập trình viên định sử dụng nó như là một biến tạm thời hoặc biến trung gian nhưng sau đó lại không cần dùng. Lập trình viên này đã quên xóa các câu lệnh khai báo và gán giá trị cho biến này.

Huang đã giới thiệu một phương pháp để xác định những bất thường trong việc sử dụng các biến dữ liệu bằng cách sử dụng sơ đồ chuyển trạng thái ứng với mỗi biến dữ liệu của chương trình. Các thành phần của sơ đồ chuyển trạng thái của một chương trình ứng với mỗi biến gồm:

- Các trạng thái, gồm:
 - U : biến chưa được gán giá trị
 - D: biến đã được gán giá trị nhưng chưa được sử dụng
 - R: biến đã được sử dụng
 - A: trạng thái lỗi
- Các hành động, gồm:
 - d: biến được gán giá trị
 - r: biến được sử dụng
 - u: biến chưa được gán giá trị hoặc được khai báo lại và chưa được gán giá trị

Hình 8.3 mô tả sơ đồ chuyển trạng thái của một biến trong một chương trình/đơn vị chương trình. Ban đầu, biến này đã được khai báo và chưa được gán giá trị nên trạng thái của chương trình là U. Tại trạng thái này, nếu biến này được sử dụng (hành động r) thì chương trình có vấn đề và trạng thái của chương trình là A. Ngược lại, trạng thái U vẫn được giữ nguyên nếu các câu lệnh tiếp theo vẫn chưa chứa lệnh gán giá trị cho biến này (hành động u). Cho đến khi gặp câu lệnh gán giá trị cho biến này (hành động d), trạng thái của chương trình được chuyển thành D. Nếu biến này được sử dụng ở các câu lệnh tiếp theo (hành động r) thì trạng thái của chương trình chuyển thành R. Ngược lại, nếu các câu lệnh tiếp theo lại gán lại giá trị cho biến (hành động d) hoặc khai báo lại biến này và không gán giá trị cho nó (hành động u) thì xảy ra vấn đề và trạng thái của chương trình là A. Tại trạng thái này, mọi hành động (d, u và r) xảy ra đều không thay đổi trạng thái của chương trình. Tại trạng thái R, nếu biến này vẫn tiếp tục được sử dụng

ở các lệnh tiếp theo (hành động r) thì trạng thái của chương trình vẫn không thay đổi. Ngược lại, nếu xuất hiện câu lệnh gán lại giá trị cho biến (hành động d) thì trạng thái của chương trình quay về D . Trong trường hợp xuất hiện câu lệnh khai báo lại biến này và không gán giá trị cho nó (hành động u) thì chương trình được chuyển từ trạng thái R sang trạng thái U .



Hình 8.3: Sơ đồ chuyển trạng thái của một biến.

Như vậy, các vấn đề với dòng dữ liệu thuộc loại 1 ứng với trường hợp dd xảy ra trong sơ đồ chuyển trạng thái. Các vấn đề thuộc loại 2 ứng với trường hợp ur và loại 3 ứng với trường hợp du . Để phát hiện các vấn đề này, chúng ta sẽ tiến hành xây dựng sơ đồ chuyển trạng thái ứng với mỗi biến như Hình 8.3. Nếu trạng thái A xuất hiện thì chương trình có vấn đề về dòng dữ liệu. Trong trường hợp này, chúng ta cần kiểm tra lại mã nguồn, tìm nguyên nhân của tình huống này và sửa lỗi. Tuy nhiên, cho dù trạng thái lỗi (trạng thái A) không xuất hiện trong quá trình phân tích chương trình, chúng ta vẫn không đảm bảo được rằng chương trình không còn lỗi. Các lỗi có thể xảy ra trong quá trình gán/gán lại giá trị cho các biến và trong quá trình sử dụng chúng (trong các câu lệnh tính toán, trong các biểu thức điều kiện, ...). Để phát hiện những lỗi này, chúng ta cần phương pháp kiểm thử dòng dữ liệu động. Phương pháp này sẽ được trình bày chi tiết ở các mục tiếp theo.

8.2 Tổng quan về kiểm thử dòng dữ liệu

Kiểm thử dòng dữ liệu tĩnh không đảm bảo phát hiện tất cả các lỗi liên quan đến việc khởi tạo, gán giá trị mới và sử dụng các biến (trong các câu lệnh tính toán và các biểu thức điều kiện như trong các lệnh rẽ nhánh và lặp). Nó được xem như một bước tiền xử lý mã nguồn trước khi áp dụng phương pháp kiểm thử dòng dữ liệu động. Chúng ta hy vọng các lỗi còn lại (không được phát hiện bởi kiểm thử dòng dữ liệu tĩnh) sẽ được phát hiện bởi phương pháp này.

Các biến xuất hiện trong một chương trình theo một số trường hợp như: khởi tạo, gán giá trị mới, tính toán, và dùng làm điều khiển trong các biểu thức điều kiện của các lệnh rẽ nhánh và lặp. Có hai lý do chính khiến chúng ta phải tiến hành kiểm thử dòng dữ liệu của chương trình:

- Chúng ta cần chắc chắn rằng một biến phải được gán đúng giá trị, tức là chúng ta phải xác định được một đường đi của biến từ một điểm bắt đầu nơi nó được định nghĩa đến điểm mà biến đó được sử dụng. Mỗi khi chưa có các ca kiểm thử để kiểm tra đường đi này, chúng ta không thể tự tin khẳng định là biến này đã được gán giá trị đúng.
- Ngay cả khi gán đúng giá trị cho biến thì các giá trị được sinh ra chưa chắc đã chính xác do tính toán hoặc các biểu thức điều kiện sai (biến được sử dụng sai).

Để áp dụng phương pháp kiểm thử dòng dữ liệu động, chúng ta phải xác định các đường dẫn chương trình có một điểm đầu vào và một điểm đầu ra sao cho nó bao phủ việc gán giá trị và sử dụng mỗi biến của chương trình/đơn vị chương trình cần kiểm thử. Cụ thể, chúng ta cần thực hiện các bước sau:

- Xây dựng đồ thị dòng dữ liệu của chương trình/đơn vị chương trình
- Chọn một hoặc một số tiêu chí kiểm thử dòng dữ liệu
- Xác định các đường dẫn chương trình phù hợp với tiêu chí kiểm thử đã chọn
- Lấy ra các biểu thức điều kiện từ tập các đường đi, thực hiện giải các biểu thức điều kiện để có được các giá trị đầu vào cho các ca kiểm thử tương ứng với các đường đi này và tính toán giá trị đầu ra mong đợi của mỗi ca kiểm thử.
- Thực hiện các ca kiểm thử để xác định các lỗi (có thể có) của chương trình.
- Sửa các lỗi (nếu có) và thực hiện lại tất cả các ca kiểm thử trong trường hợp bước trên phát hiện ra lỗi.

Như vậy, các bước trong quy trình kiểm thử dòng dữ liệu động cũng tương tự như các bước trong quy trình kiểm thử dòng điều khiển. Tuy nhiên, đồ thị dòng điều khiển hoàn toàn khác với đồ thị dòng dữ liệu. Hơn nữa, các tiêu chí kiểm thử (độ đo kiểm thử) ứng với hai kỹ thuật này cũng khác nhau. Kết quả là, các đường đi và phương pháp chọn chúng từ đồ thị cũng khác nhau.

8.3 Đồ thị dòng dữ liệu

Trong mục này, chúng ta sẽ tìm hiểu về đồ thị dòng dữ liệu và cách xây dựng nó từ đơn vị chương trình. Trong thực tế, chúng ta có thể sử dụng các tiện ích của các chương trình dịch để xây dựng đồ thị này một cách tự động. Đồ thị dòng dữ liệu của một chương trình/đơn vị chương trình sử dụng các khái niệm liên quan đến việc định nghĩa và sử dụng các biến.

Định nghĩa 8.1. (Định nghĩa của một biến.) Một câu lệnh thực hiện việc gán giá trị cho một biến được gọi là câu lệnh định nghĩa của biến đó (ký hiệu là *def*). Ví dụ, trong hàm *VarTypes* viết bằng ngôn ngữ C như Hình 8.4, câu lệnh *i = x* (dòng 3) là một ví dụ về việc định nghĩa của biến *i*.

Định nghĩa 8.2. (Chưa định nghĩa một biến.) Một câu lệnh khai báo một biến nhưng chưa thực hiện việc gán giá trị được gọi là *undef* với biến đó. Ví dụ, trong hàm *VarTypes* viết bằng ngôn ngữ C như Hình 8.4, câu lệnh *iptr = malloc(sizeof(int));* (dòng 4) là một ví dụ về việc chưa định nghĩa biến *iptr*. Câu lệnh tiếp theo (**iptr = i + x;* (dòng 5)) là một định nghĩa của biến này. Tuy nhiên, câu lệnh ở dòng 9 lại định nghĩa lại biến *iptr* và vì vậy câu lệnh này là một ví dụ khác về việc chưa định nghĩa biến này.

Định nghĩa 8.3. (Biến được sử dụng.) Một câu lệnh sử dụng một biến (để tính toán hoặc để kiểm tra các điều kiện) được gọi là *use* của biến đó.

Định nghĩa 8.4. (Biến được sử dụng để tính toán.) Một câu lệnh sử dụng một biến để tính toán giá trị của một biến khác được gọi là *c-use* với biến đó. Ví dụ, trong hàm *VarTypes* như Hình 8.4, câu lệnh **iptr = i + x;* (dòng 5) là *c-use* ứng với biến *i* và biến *x*.

Định nghĩa 8.5. (Biến được sử dụng để kiểm tra các điều kiện.) Một câu lệnh sử dụng một biến trong các biểu thức điều kiện (câu lệnh rẽ nhánh, lặp, ...) được gọi là *p-use* với biến đó. Ví dụ, trong hàm *VarTypes* như Hình 8.4, câu lệnh *if (*iptr > y) ...* (dòng 6) là *p-use* ứng với biến *iptr* và biến *y*.

```

int VarTypes(int x, int y){
1.  int i;
2.  int *iptr;
3.  i = x;
4.  iptr = malloc(sizeof(int));
5.  *iptr = i + x;
6.  if (*iptr > y)
7.      return (x);
8.  else {
9.      iptr = malloc(sizeof(int));
10.     *iptr = x + y;
11.     return(*iptr);
12. } //end if
} //the end

```

Hình 8.4: Ví dụ về định nghĩa và sử dụng các biến.

Định nghĩa 8.6. (Đồ thị dòng dữ liệu.) Đồ thị dòng dữ liệu của một chương trình/đơn vị chương trình là một đồ thị có hướng $G = \langle N, E \rangle$, với:

N là tập các đỉnh tương ứng với các câu lệnh def hoặc c-use của các biến được sử dụng trong đơn vị chương trình. Đồ thị G có hai đỉnh đặc biệt là đỉnh bắt đầu (tương ứng với lệnh def của các biến tham số) và đỉnh kết thúc đơn vị chương trình.

- E là tập các cạnh tương ứng với các câu lệnh p-use của các biến.

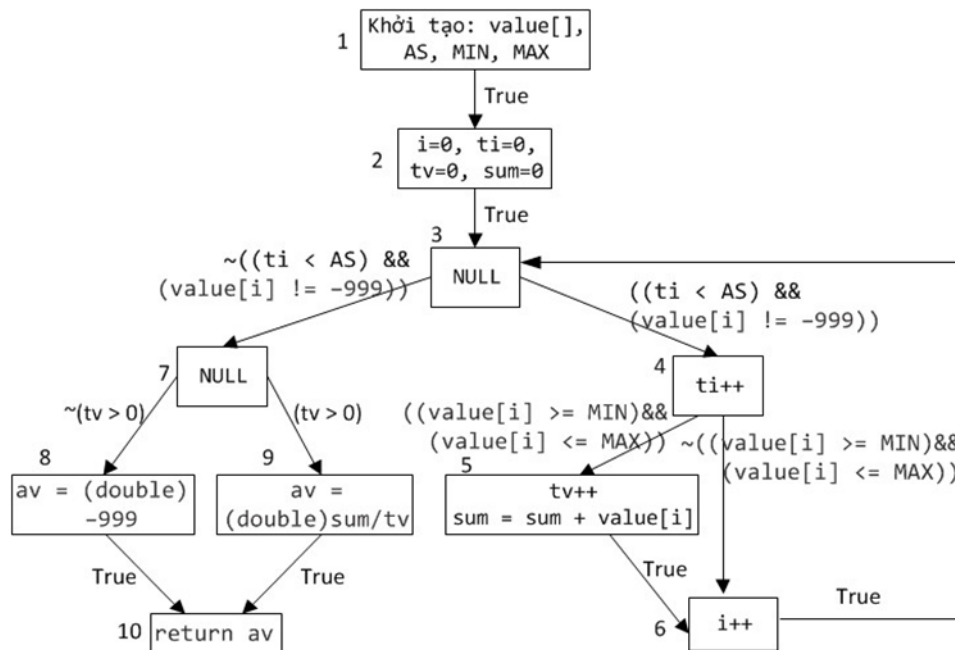
```

double ReturnAverage(int value[], int AS, int MIN, int MAX){
    int i, ti, tv, sum;
    double av;
    i = 0; ti = 0; tv = 0; sum = 0;
    while (ti < AS && value[i] != -999) {
        ti++;
        if (value[i] >= MIN && value[i] <= MAX) {
            tv++;
            sum = sum + value[i];
        }
        i++;
    } //end while
    if (tv > 0)
        av = (double)sum/tv;
    else
        av = (double) -999;
    return (av);
} //the end

```

Hình 8.5: Mã nguồn của hàm ReturnAverage bằng ngôn ngữ C.

Chúng ta sẽ tìm hiểu cách xây dựng đồ thị dòng dữ liệu của một đơn vị chương trình thông qua ví dụ về hàm ReturnAverage bằng ngôn ngữ C trong Hình 8.5. Đồ thị dòng dữ liệu của hàm này được trình bày ở Hình 8.6.



Hình 8.6: Đồ thị dòng dữ liệu của hàm ReturnAverage trong Hình 8.5.

Trong đồ thị này, đỉnh bắt đầu (đỉnh 1) thể hiện việc định nghĩa (def) các biến tham số value[], AS, MIN, MAX. Đỉnh 2 tương ứng với việc định nghĩa các biến cục bộ i, ti, tv, sum. Tiếp đến, chúng ta sử dụng đỉnh NULL (đỉnh 3) với mục đích đây là điểm bắt đầu của vòng lặp while. Chú ý rằng đỉnh NULL không thực hiện bất cứ tính toán hay định nghĩa cho biến nào cả. Đỉnh này cũng sẽ là điểm kết thúc của vòng lặp này sau khi biểu thức điều kiện của vòng lặp là sai. Câu lệnh ti++; được biểu diễn bằng đỉnh 4. Cạnh (3, 4) biểu diễn biểu thức điều kiện của vòng lặp (((ti < AS) && (value[i] != -999))). Ý nghĩa của cạnh này là nếu biểu thức này đúng thì chuyển từ đỉnh 3 sang thực hiện đỉnh 4. Các câu lệnh tv++; và sum = sum + value[i]; được biểu diễn bởi đỉnh 5. Để chuyển từ đỉnh 4 sang đỉnh 5 (cạnh (4, 5)), điều kiện tương ứng với cạnh này phải đúng (((value[i] >= MIN) && (value[i] <= MAX)) là đúng). Tiếp theo, đỉnh 6 biểu diễn câu lệnh i++;. Trong trường hợp biểu thức sau là sai ((value[i] >= MIN) && (value[i] <= MAX)), đồ thị sẽ chuyển từ đỉnh 4 sang đỉnh 6 (cạnh (4, 6)). Từ đỉnh 5, đồ thị sẽ chuyển đến đỉnh 6 (cạnh (5, 6)) vì không có biểu thức điều kiện của cạnh này. Vòng lặp while sẽ kết thúc khi điều kiện ((ti < AS) && (value[i] != -999)) là sai. Khi đó, đồ thị sẽ chuyển từ đỉnh 3 sang đỉnh 7 (cạnh (3, 7)). Đỉnh 7 là đỉnh NULL tương ứng với câu lệnh if (tv > 0). Đỉnh 8 là biểu diễn của câu lệnh av = (double) -999; trong khi câu lệnh av = (double) sum/tv; được biểu diễn bằng đỉnh 9. Từ đỉnh 7, nếu điều kiện tv > 0 là đúng thì đồ thị chuyển sang đỉnh 9 (cạnh (7, 9)). Ngược lại, đồ thị chuyển từ đỉnh 7 sang đỉnh 8 (cạnh (7, 8)). Cuối cùng, đỉnh 10 được sử dụng để biểu diễn câu lệnh return(av);. Từ các

đỉnh 8 và 9, đồ thị chuyển đến đỉnh 10 vì không có biểu thức điều kiện cho các cạnh (8, 10) và (9, 10).

8.4 Các khái niệm về dòng dữ liệu

Sau khi xây dựng đồ thị dòng dữ liệu của đơn vị chương trình, chúng ta cần xác định các đường đi của đơn vị chương trình của mỗi biến dữ liệu ứng với các độ đo kiểm thử. Trong mỗi đường dẫn này, biến dữ liệu được định nghĩa tại một đỉnh nào đó và được sử dụng tại các câu lệnh tiếp theo ứng với các đỉnh hoặc các cạnh của đường đi này. Trong mục này, chúng ta sẽ tìm hiểu một số khái niệm cơ bản về dòng dữ liệu. Các khái niệm này sẽ được sử dụng để phục vụ mục đích trên.

Định nghĩa 8.8. (Global c-use.) Giả sử biến x được sử dụng để tính toán (c-use) tại đỉnh i của đồ thị dòng dữ liệu. Việc sử dụng biến x tại đỉnh i được gọi là Global c-use nếu x đã được định nghĩa ở các đỉnh trước đó.

Ví dụ: Trong đồ thị dòng dữ liệu của hàm ReturnAverage được mô tả ở Hình 8.5, việc sử dụng biến tv tại đỉnh 9 là Global c-use vì biến này đã được định nghĩa tại các đỉnh 2 và 5.

Định nghĩa 8.8. (Def -clear path.) Giả sử biến x được định nghĩa (def) tại đỉnh i và được sử dụng tại đỉnh j . Một đường đi từ i đến j ký hiệu là $(i - n_1 - \dots - n_m - j)$ với $m \geq 0$ được gọi là Def -clear path ứng với biến x nếu biến này không được định nghĩa tại các đỉnh từ n_1 đến n_m .

Ví dụ: Trong đồ thị dòng dữ liệu của hàm ReturnAverage được mô tả ở Hình 8.6, đường đi $(2 - 3 - 4 - 5)$ là một Def -clear path ứng với biến tv vì biến này được định nghĩa tại đỉnh 2, được sử dụng tại đỉnh 5, và không được định nghĩa tại các đỉnh 3 và 4. Tương tự, đường đi $(2 - 3 - 4 - 6 - 3 - 4 - 6 - 3 - 4 - 5)$ cũng là một Def -clear path ứng với biến tv .

Định nghĩa 8.9. (Global def.) Một đỉnh i được gọi là Global def của biến x nếu đỉnh này định nghĩa biến x (def) và có một Def -clear path của x từ đỉnh i tới đỉnh chứa một Global c-use hoặc cạnh chứa một p-use của biến này.

Bảng 8.1 liệt kê tất cả các Global def và Global c use xuất hiện trong đồ thị dòng dữ liệu ở Hình 8.6. Trong bảng này, def(i) được sử dụng để chỉ tập các biến có Global def tại đỉnh i . Tương tự, c-use(i) chỉ tập các biến có Global c-use tại đỉnh i . Tất cả các điều

kiện ứng với các cạnh và các p-use xuất hiện trong đồ thị dòng dữ liệu ở Hình 8.6 cũng được liệt kê trong Bảng 4.2, với $p\text{-use}(i, j)$ là tập các biến có p-use tại cạnh (i, j) .

Định nghĩa 8.10. (Simple path.) Một đường đi trong đồ thị dòng dữ liệu được gọi là một Simple path nếu các đỉnh chỉ xuất hiện đúng một lần trừ đỉnh đầu và đỉnh cuối.

Ví dụ: Trong đồ thị dòng dữ liệu của hàm ReturnAverage được mô tả ở Hình 8.6, các đường đi $(2 - 3 - 4 - 5)$ và $(3 - 4 - 6 - 3)$ là các Simple paths.

Định nghĩa 8.11. (Loop-free path.) Một đường đi trong đồ thị dòng dữ liệu được gọi là một Loop-free path nếu các đỉnh chỉ xuất hiện đúng một lần.

Định nghĩa 8.12. (Complete-path.) Một đường đi được gọi là một Complete-path nếu nó có điểm bắt đầu và điểm kết thúc chính là điểm bắt đầu và điểm kết thúc của đồ thị dòng dữ liệu.

Định nghĩa 8.13. (Du-path.) Một đường đi $(n_1 - n_2 - \dots - n_j - n_k)$ được gọi là một Du-path (definition-use path) ứng với biến x nếu đỉnh n_1 là Global def của biến x và:

- đỉnh n_k có một Global c-use với biến x và $(n_1 - n_2 - \dots - n_j - n_k)$ là một Def-clear simple path với biến x , hoặc
- cạnh (n_j, n_k) có p-use với biến x và $(n_1 - n_2 - \dots - n_j)$ là Def-clear loop-free path với biến này.

Ví dụ: Trong đồ thị dòng dữ liệu của hàm ReturnAverage (Hình 8.6), đường đi $(2 - 3 - 4 - 5)$ là một Du-path ứng với biến tv vì đỉnh 2 là Global def của biến tv , đỉnh 5 có Global c-use với biến này và $(2 - 3 - 4 - 5)$ là một Def-clear simple path với biến tv . Một ví dụ khác, đường đi $(2 - 3 - 7 - 9)$ cũng là một Du-path ứng với biến tv vì đỉnh 2 là Global def của biến này, cạnh $(7, 9)$ có p-use với biến tv và $(2 - 3 - 7)$ là một Def-clear loop-free path với tv .

8.5 Các độ đo cho kiểm thử dòng dữ liệu

Các độ đo hay các tiêu chí kiểm thử dòng dữ liệu là đầu vào cùng với đồ thị dòng dữ liệu nhằm xác định các đường đi cho mục đích kiểm thử tương ứng. Trong mục này, chúng ta sẽ tìm hiểu các độ đo phổ biến đang được sử dụng trong kiểm thử dòng dữ liệu. Trước hết ta có ba độ đo đơn giản là All-defs, All-c-uses, và All-p-uses tương ứng với tất cả các đường đi, đường đi qua tất cả các đỉnh, và đường đi qua tất cả các cạnh. Các độ đo tiếp theo sẽ giả sử ta đã xác định được các đỉnh xác định và sử dụng và các Du-path cho từng biến. Các định nghĩa sau cũng giả sử T là một tập hợp các (đoạn) đường

trong đồ thị dòng dữ liệu $G = \langle N, E \rangle$ và V là tập các biến được sử dụng trong đơn vị chương trình.

All-defs: Mỗi một biến $x \in V$ và mỗi đỉnh $i \in N$, giả sử x có một Global def tại i , chọn một Complete-path chứa một Def-clear path từ đỉnh i tới đỉnh j sao cho tại j có chứa một Global c-use của x , hoặc cạnh (j, k) chứa một p-use của biến x . Điều này có nghĩa là đối với mỗi một định nghĩa (def) của x tại một đỉnh ta cần ít nhất một đường đi xuất phát từ đỉnh đó tới một đỉnh khác sử dụng biến x sao cho đường đi này chứa một Def-clear path của biến đó và thuộc về một Complete-path nào đó.

Ví dụ: Biến tv có hai Global def tại các đỉnh 2 và 5 (Hình 8.6, Bảng 8.1 và Bảng 8.2). Trước hết, ta quan tâm đến Global def tại đỉnh 2. Chúng ta thấy rằng có một Global c-use của biến tv tại đỉnh 5 và tồn tại một Def-clear path $(2 - 3 - 4 - 5)$ từ đỉnh 2 tới đỉnh 5. Ta cũng có được một Complete-path $(1 - 2 - 3 - 4 - 5 - 6 - 3 - 7 - 9 - 10)$ chứa đường đi này. Do vậy ta kết luận $(2 - 3 - 4 - 5)$ thỏa mãn độ đo All-defs. Tương tự, đường đi $(2 - 3 - 7 - 8)$ của biến tv cũng thỏa mãn độ đo All-defs do có một Global def tại đỉnh 2, có một p-use tại cạnh $(7, 8)$, có một Def-clear path là $(2 - 3 - 7 - 8)$ từ đỉnh 2 tới cạnh $(7, 8)$ và nó thuộc về một Complete-path $(1 - 2 - 3 - 7 - 8 - 10)$. Quan tâm đến Global def tại đỉnh 5 và có một Global c-use tại đỉnh 9, $(5 - 6 - 3 - 7 - 9)$ là một Def-clear path và tồn tại một Complete-path là $(1 - 2 - 3 - 4 - 5 - 6 - 3 - 7 - 9 - 10)$ chứa đường đi này. Vì vậy $(5 - 6 - 3 - 7 - 9)$ cũng thỏa mãn độ đo All-defs. Để quá trình kiểm thử dòng dữ liệu thỏa mãn độ đo All-defs, chúng ta cần tiến hành một cách tương tự với các biến còn lại.

All-c-uses: Với mỗi một biến x và mỗi đỉnh i sao cho i là Global def với biến x , chọn các Complete-path bao gồm các Def-clear path từ đỉnh i tới tất cả các đỉnh j sao cho j là Global c-use của x . Điều này có nghĩa là cứ mỗi định nghĩa (def) của x ta tìm tất cả các đường đi xuất phát từ def của x tới tất cả các c-use của biến x sao cho các đường đi này có chứa một Def-clear path của x và thuộc về một Complete-path nào đó.

Ví dụ: Ta sẽ tìm tất cả các đường đi thỏa mãn độ đo All-c-uses ứng với biến ti (Hình 8.6, Bảng 8.1, và Bảng 8.2). Chúng ta tìm thấy hai Global def của biến này tại các đỉnh 2 và 4. Với đỉnh 2, có một Global c-use của biến ti tại đỉnh 4. Tuy nhiên, với Global def của biến này tại đỉnh 4, ta không tìm thấy Global c-use nào của biến ti . Từ đỉnh 2, ta có một Def-clear path tới đỉnh 4 là $(2 - 3 - 4)$. Chúng ta có thể tìm thấy bốn Complete-path từ đồ thị dòng dữ liệu (Hình 8.6) chứa đường đi này như sau:

- (1 - 2 - 3 - 4 - 5 - 6 - 3 - 7 - 8 - 10),
- (1 - 2 - 3 - 4 - 5 - 6 - 3 - 7 - 9 - 10),
- (1 - 2 - 3 - 4 - 6 - 3 - 7 - 8 - 10), và
- (1 - 2 - 3 - 4 - 6 - 3 - 7 - 9 - 10).

Chúng ta có thể chọn một hoặc một số trong bốn đường đi trên để đảm bảo độ đo này ứng với biến ti. Để quá trình kiểm thử dòng dữ liệu thỏa mãn độ đo All-c-uses, chúng ta cần tiến hành một cách tương tự với các biến còn lại (i, tv, và sum).

All-p-uses: Với mỗi một biến x và mỗi đỉnh i sao cho i là Global def với biến x, chọn các Complete-path bao gồm các Def-clear path từ đỉnh i tới tất cả các cạnh (j, k) sao cho có một p-use của x tại cạnh này. Điều này có nghĩa là cứ mỗi định nghĩa (def) của x ta tìm tất cả các đường đi xuất phát từ def của x tới tất cả các p-use của biến đó sao cho các đường đi này có chứa một Def-clear path của x và thuộc về một Complete-path nào đó.

Ví dụ: Ta sẽ tìm tất cả các đường đi thỏa mãn độ đo All-p-uses ứng với biến tv (Hình 8.6, Bảng 8.1, và Bảng 8.2). Biến tv có hai Global def tại các đỉnh 2 và 5. Tại đỉnh 2, ta có hai p-use của biến này tại các cạnh (7, 8) và (7, 9). Dễ thấy (2 - 3 - 7 - 8) là một Def-clear path của tv từ đỉnh 2 đến cạnh (7, 8) và (2 - 3 - 7 - 9) là một Def-clear path của tv từ đỉnh 2 đến cạnh (7, 9). Tương tự, (5 - 6 - 3 - 7 - 8) là một Def-clear path của tv từ đỉnh 5 đến cạnh (7, 8) và (5 - 6 - 3 - 7 - 9) là một Def-clear path của tv từ đỉnh 5 đến cạnh (7, 9). Chúng ta có thể tìm thấy bốn Complete-path từ đồ thị dòng dữ liệu (Hình 7.6) chứa các đường đi này như sau:

- (1 - 2 - 3 - 7 - 8 - 10),
- (1 - 2 - 3 - 7 - 9 - 10),
- (1 - 2 - 3 - 4 - 5 - 6 - 3 - 7 - 8 - 10), và
- (1 - 2 - 3 - 4 - 5 - 6 - 3 - 7 - 9 - 10).

All-p-uses/Some-c-uses: Độ đo này tương tự như độ đo All-p-uses ngoại trừ trường hợp khi có một nghĩa (def) của biến x mà không có một p-use của biến này. Trong trường hợp này, ta sử dụng độ đo Some-c-uses được định nghĩa như sau.

Định nghĩa 8.14. (Some-c-uses.) Với mỗi một biến x và mỗi đỉnh i sao cho i là Global def với biến x, chọn các Complete-path bao gồm các Def-clear path từ đỉnh i tới một số đỉnh j sao cho j là Global c-use của x.

Ví dụ: Ta sẽ tìm tất cả các đường đi thỏa mãn độ đo All-p-uses/Some- c-uses ứng với biến i (Hình 8.6, Bảng 8.1, và Bảng 8.2). Biến i có hai Global def tại các đỉnh 2 và đỉnh 6. Dễ thấy rằng không có p-use của biến này (Hình 8.6). Vì vậy, ta quan tâm đến độ đo Some-c-uses của biến i . Tại đỉnh 2, có một Global c-use của i tại đỉnh 6 và có một Def-clear path (2 - 3 - 4 - 5 - 6). Vì vậy, để thỏa mãn độ đo All-p-uses/Some-c-uses với biến này, ta chọn Complete-path (1 - 2 - 3 - 4 - 5 - 6 - 3 - 7 - 9 - 10) chứa đường đi trên.

All-c-uses/Some-p-uses: Độ đo này tương tự như độ đo All-c-uses ngoại trừ trường hợp khi có một nghĩa (def) của biến x mà không có một Global c-use của biến này. Trong trường hợp này, ta sử dụng độ đo Some-p-uses được định nghĩa như sau.

Định nghĩa 8.15. (Some-p-uses.) Với mỗi một biến x và mỗi đỉnh i sao cho i là Global def với biến x , chọn các Complete-path bao gồm các Def-clear path từ đỉnh i tới một số cạnh (j, k) sao cho có một p-use của x tại cạnh này.

Ví dụ: Ta sẽ tìm tất cả các đường đi thỏa mãn độ đo All-c-uses/Some- p-uses ứng với biến AS (Hình 8.6, Bảng 8.1, và Bảng 8.2). Biến AS chỉ có một Global def tại đỉnh 1. Dễ thấy rằng không có Global c-use của biến này (Hình 4.13). Vì vậy, ta quan tâm đến độ đo Some-p-uses của biến AS. Từ đỉnh 1, có các p-use của AS tại các cạnh (3, 7) và (3, 4) và có các Def-clear path tương ứng với hai cạnh này là (1 - 2 - 3 - 7) và (1 - 2 - 3 - 4). Có rất nhiều Complete-path chứa hai đường đi này. Ví dụ (1 - 2 - 3 - 4 - 5 - 6 - 3 - 7 - 9 - 10).

All-uses: Độ đo này bao gồm các đường đi được sinh ra từ các độ đo All- p-uses và All-c-uses (như đã định nghĩa ở trên). Điều này có nghĩa là với mỗi việc sử dụng (c-use hoặc p-use) của một biến thì có một đường đi từ định nghĩa (def) của biến đó tới các sử dụng của nó.

All-du-paths: Với mỗi một biến x và mỗi đỉnh i sao cho i là Global def với biến x , chọn các Complete-path chứa các tất cả các Du-path từ đỉnh i tới:

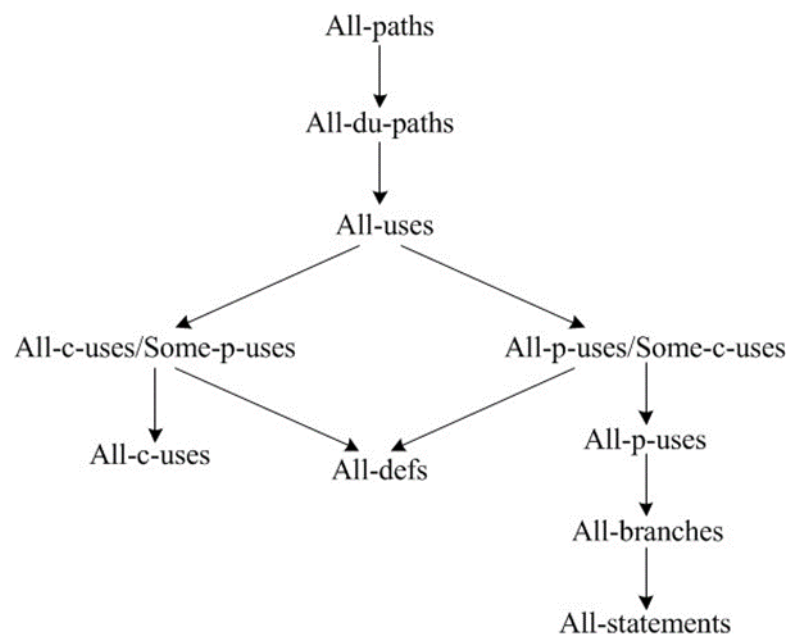
- tất cả các đỉnh j sao cho có một Global c-use của biến x tại j , và
- tất cả các cạnh (j, k) sao cho có một p-use của biến x tại (j, k) .

So sánh các độ đo của kiểm thử dòng dữ liệu: Sau khi tìm hiểu các độ đo về kiểm thử dòng dữ liệu, chúng ta cần so sánh mối quan hệ giữa chúng để trả lời câu hỏi “chúng ta nên chọn độ đo nào cho kiểm thử dòng dữ liệu?”. Với mỗi cặp độ đo ở trên, chúng có thể so sánh với nhau hoặc không. Rapps và Weyuker đã định nghĩa mối quan hệ “bao gồm” giữa hai độ đo như sau.

Định nghĩa 8.16. (Mối quan hệ bao gồm.) Cho hai độ đo $c1$ và $c2$, ta nói $c1$ bao gồm $c2$ nếu mọi đường đi đầy đủ (Complete-paths) sinh ra từ đồ thị dòng dữ liệu thỏa mãn $c1$ thì cũng thỏa mãn $c2$.

Định nghĩa 8.17. (Mối quan hệ bao gồm chặt.) Cho hai độ đo $c1$ và $c2$, ta nói $c1$ bao gồm chặt $c2$, ký hiệu là $c1 \rightarrow c2$, nếu $c1$ bao gồm $c2$ và tồn tại một số đường đi đầy đủ sinh ra từ đồ thị dòng dữ liệu thỏa mãn $c2$ nhưng không thỏa mãn $c1$.

Để thấy mối quan hệ bao gồm chặt ($\rightarrow$) có tính bắc cầu. Hơn nữa, với hai độ đo $c1$ và $c2$, có thể $c1$ không bao gồm chặt $c2$ và $c2$ cũng không bao gồm chặt $c1$. Trong trường hợp này ta nói hai độ đo này là không so sánh được. Frankl và Weyuker đã tổng kết các mối quan hệ giữa các độ đo cho kiểm thử dòng dữ liệu trong Hình 8.6. Với các độ đo như đã trình bày ở trên, All-du-paths là độ đo tốt nhất. Độ đo này bao gồm chặt độ đo All-uses. Tương tự, độ đo All-uses bao gồm chặt hai độ đo All-p-uses/Some-c-uses và All-c-uses/Some-p-uses trong khi chúng bao gồm chặt độ đo All-defs. Hơn nữa, độ đo All-p-uses/Some-c-uses bao gồm chặt độ đo All-p-uses trong khi độ đo All-c-uses/Some-p-uses bao gồm chặt độ đo All-c-uses. Tuy nhiên, chúng ta không thể tìm thấy mối quan hệ bao gồm chặt giữa hai độ đo All-c-uses và All-p-uses.



Hình 8.7: Mối quan hệ giữa các độ đo cho kiểm thử dòng dữ liệu.

8.6 Sinh các ca kiểm thử

Để tiến hành phương pháp kiểm thử dòng dữ liệu, trước hết chúng ta phải sinh đồ thị dòng dữ liệu của đơn vị chương trình. Với độ đo kiểm thử C , chúng ta sẽ xác định tất cả các đường đi đầy đủ (Complete-paths) thỏa mãn độ đo này. Tuy nhiên, không phải

đường đi nào cũng có thể tìm được một bộ dữ liệu đầu vào để nó được thực thi khi chạy đơn vị chương trình. Nếu tồn tại một bộ dữ liệu đầu vào như vậy thì đường đi tương ứng được gọi là đường đi thực thi được. Như vậy, bài toán còn lại hiện nay là làm thế nào để sinh được bộ đầu vào cho từng đường đi đầy đủ trên. Bộ đầu vào này cùng với giá trị đầu ra mong đợi sẽ là ca kiểm thử cho đường đi này. Để trả lời câu hỏi này, ta xét ví dụ sau.

Ví dụ: Xét đường đi đầy đủ (1 - 2 - 3 - 4 - 5 - 6 - 3 - 7 - 9) từ đồ thị dòng dữ liệu trong Hình 8.6. Từ đường đi này, ta sẽ xác định các biểu thức thuộc các p-uses nằm trên các cạnh của nó. Cụ thể, ta có các biểu thức sau:

1. $((ti < AS) \ \&\& \ (value[i] \neq -999))$ (thuộc cạnh (3, 4)),
2. $((value[i] \geq MIN) \ \&\& \ (value[i] \leq MAX))$ (thuộc cạnh (4, 5)),
3. $\sim((ti < AS) \ \&\& \ (value[i] \neq -999))$ (thuộc cạnh (3, 7)), và
4. $tv > 0$ (thuộc cạnh (7, 9))

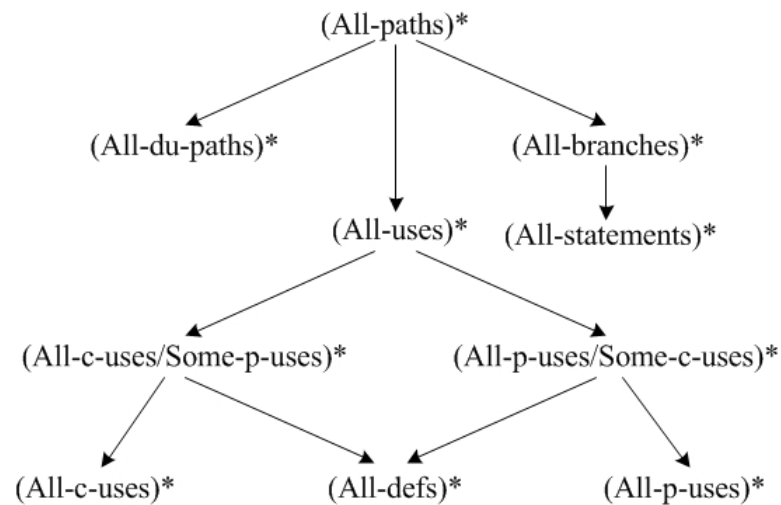
Chúng ta sẽ sinh ra các giá trị đầu vào và kiểm tra các giá trị đó sao cho thỏa mãn hết các điều kiện trên. Chú ý, các giá trị đầu vào ở đây là các tham số của hàm ReturnAverage (Hình 4.12), giá trị của biến i ở điều kiện (3) lớn hơn 1 so với giá trị nó tại các biểu thức (1) và (2), và giá trị của biến ti ở các điều kiện (2) và (3) lớn hơn 1 so với giá trị nó tại biểu thức (1). Ví dụ, bộ đầu vào $([1, -999], 2, 1, 2)$ thỏa mãn tất cả các điều kiện trên.

Để lựa chọn các bộ đầu vào ứng với các đường đi đầy đủ thỏa mãn một độ đo cho trước, chúng ta phải đảm bảo rằng các đường đi này là thực thi được. Giả sử PC là tập các đường đi đầy đủ của đơn vị chương trình ứng với độ đo C. Độ đo này chỉ có ích khi tồn tại ít nhất một đường đi thuộc PC sao cho nó thực thi được. Trong trường hợp này, chúng ta gọi C là độ đo dòng dữ liệu thực thi được, ký hiệu là (C). Ví dụ, (All-c-uses) được sử dụng để chỉ tất cả các đường đi đầy đủ thỏa mãn độ đo All-c-uses sao cho chúng thực thi được. Cụ thể, (All-c-uses)* được định nghĩa như sau.

Định nghĩa 8.18. ((All-c-uses) .) Với mỗi một biến x và mỗi đỉnh i sao cho i là Global def với biến x , chọn các đường đi đầy đủ thực thi được bao gồm các Def-clear path từ đỉnh i tới tất cả các đỉnh j sao cho j là Global c-use của x .

Tương tự, chúng ta có thể định nghĩa (All-paths), (All-du-paths), (All-uses), (All-p-uses), (All-p-uses/Some-c-uses), (All-c-uses/Some-p-uses), (All-defs), (All-branches),

và (All-statements). Mỗi quan hệ bao gồm chặt giữa từng cặp này đã được Frankl và Weyuker đã tổng kết trong hình 8.8



Hình 8.8: Mối quan hệ giữa các độ đo dòng dữ liệu thực thi được

Bài tập cuối bài:

1. So sánh kiểm thử hộp trắng và kiểm thử hộp đen?
2. Trình bày các vấn đề phổ biến về dòng dữ liệu?
3. Trình bày các khái niệm cơ bản về dòng dữ liệu?
4. Trình bày các độ đo cho kiểm thử dòng dữ liệu?
5. Các bài tập thực hành:

Bài 1. Cho đoạn mã nguồn như sau:

```

double ReturnAverage(int value[], int AS, int MIN, int MAX){
    int i, ti, tv, sum;
    double av;
    i = 0; ti = 0; tv = 0; sum = 0;
    while (ti < AS && value[i] != -999) {
        ti++;
        if (value[i] >= MIN && value[i] <= MAX) {
            tv++;
            sum = sum + value[i];
        }
        i++;
    } //end while
    if (tv > 0)
        av = (double)sum/tv;
    else
        av = (double) -999;
    return (av);
} //the end
  
```

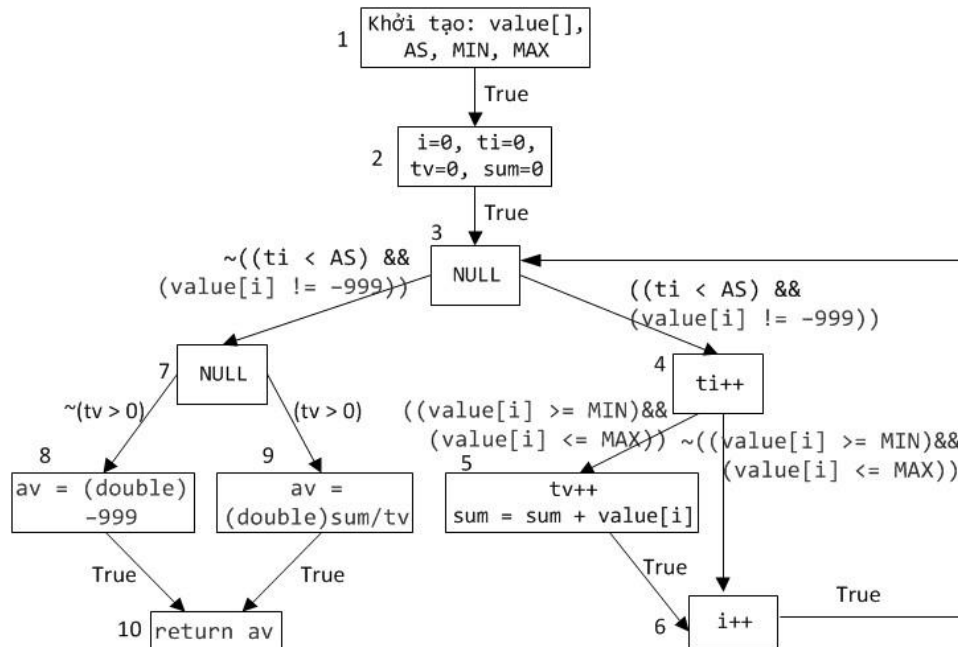
1. Vẽ đồ thị dòng dữ liệu cho đoạn mã nguồn trên

2. Xác định các def và c-use của các đỉnh

3. Xác định các Complete-path từ đồ thị dòng dữ liệu

Hướng dẫn:

1. Vẽ đồ thị dòng dữ liệu:



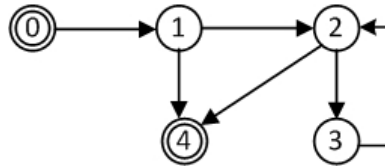
2. Xác định các def và c-use của các đỉnh

Đỉnh i	$def(i)$	$c-use(i)$
1	{value, AS, MIN, MAX}	{}
2	{i, ti, tv, sum}	{}
3	{}	{}
4	{ti}	{ti}
5	{tv, sum}	{tv, i, sum, value}
6	{i}	{i}
7	{}	{}
8	{av}	{}
9	{av}	{sum, tv}
10	{}	{av}

3. Xác định các Complete-path từ đồ thị dòng dữ liệu

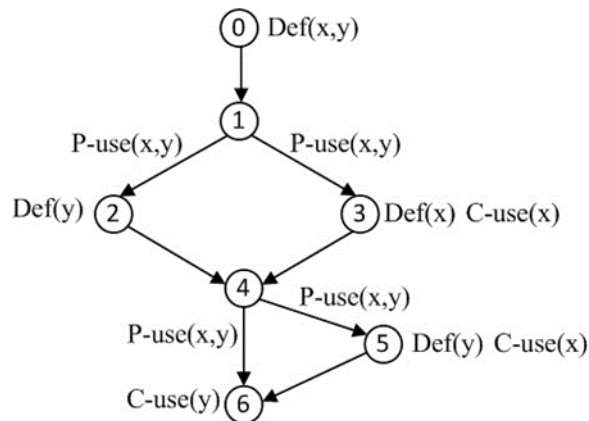
- (1 - 2 - 3 - 4 - 5 - 6 - 3 - 7 - 8 - 10)
- (1 - 2 - 3 - 4 - 6 - 3 - 7 - 8 - 10)
- (1 - 2 - 3 - 4 - 6 - 3 - 7 - 8 - 10)
- (1 - 2 - 3 - 4 - 6 - 3 - 7 - 9 - 10)

Bài 2. Cho đồ thị dòng dữ liệu như sau:



1. Hãy xác định tất cả các *Complete-path* từ đồ thị này.
2. Các đường đi (0 - 1 - 2 - 3), (1 - 2 - 3 - 2 - 4), (3 - 2 - 4), (2 - 3 - 2) và (0 - 1 - 4) có phải là các *simple-paths* không? Giải thích?
3. Các đường đi (2 - 3 - 2), (2 - 3) và (3 - 2 - 4) có phải là các *loop-free-paths* không? Giải thích?

Bài 3. Cho đồ thị dòng dữ liệu như sau:



1. Hãy xác định tất cả các *Def-clear-path* của các biến x và y.
2. Hãy xác định tất cả các *du-paths* của các biến x và y.
3. Dựa vào các chuẩn của kiểm thử dòng dữ liệu hãy xác định tất cả các *All-p-uses/Some-c-uses* và *All-c-uses/Some-p-uses*.

Bài 4. Cho đoạn lệnh như sau:

```

1 #define MAX=50
2 int intValue=0;
3 int iCount=0;
4 for(int i=0;i<MAX;i++){
5     if(i%2==0){
6         intValue = intValue+i;
7     }
8     else {
9         intValue = intValue*i;
10    } //end if-else
11 } //end for
  
```

1. Biến `i` trong vòng lặp `for` có ảnh hưởng đến giá trị của biến `intValue` không? Tại sao?
2. Hãy xác định các câu lệnh ảnh hưởng đến giá trị của biến `intValue` tại các câu lệnh 6 và 8.

Bài 5. Cho đoạn lệnh như sau:

```
int returnSumArray(int intArr[], int size){  
    int intCount, intSum;  
    intCount = 10, intSum = 0;  
    while(intCount < size){  
        intSum = intSum + intArr[intCount];  
        intCount++;  
    }//end while  
    return intSum;  
}
```

1. Hãy xây dựng hàm phân mảnh của hàm này.
2. Xây dựng đồ thị dòng dữ liệu và xác định các định nghĩa (`def`) và sử dụng (`use`) của tất cả các biến trong chương trình.